AULA 13 • BACKEND

Observabilidade e Testes

Como saber se a API funciona e o que ela está fazendo agora: testes unitários, E2E e métricas com Prometheus e Grafana

Conceito → pirâmide de testes, métricas, logs e traces • Prática → JUnit + Testcontainers + Grafana

AGENDA

O que vamos ver hoje

- **Parte 1, testes unitários:** a pirâmide de testes, JUnit, Mockito e o que vale a pena mockar.
- **Parte 2, testes E2E:** a API com PostgreSQL e RabbitMQ reais (Testcontainers) e a tela no navegador (Playwright).
- **Parte 3, observabilidade:** métricas, logs e traces; Actuator, Micrometer, Prometheus e Grafana.
- **Parte 4, na prática:** subir a stack, gerar carga na fila do RabbitMQ e ler o dashboard.

1. Testes unitários
Rápidos, sem infraestrutura

2. Testes E2E
Banco e broker reais; tela com Playwright

3. Observabilidade
Métricas, logs e traces

4. Na prática
45 testes e um dashboard ao vivo

RECAPITULANDO

De onde viemos

- Nas aulas 08 a 12 a API ganhou **banco, DTOs, Swagger, integração com o Open-Meteo e mensageria com RabbitMQ**.
- Ela funciona. Mas **como sabemos** que continua funcionando depois da próxima alteração?
- E, em produção: **o que ela está fazendo agora?** Está lenta? O Open-Meteo está falhando? A fila está acumulando?
- Hoje respondemos as duas perguntas: **testes** e **observabilidade**.



Testes: "funciona?"

Antes de entregar, de forma automática e repetível: a cada mudança, em segundos.



Observabilidade: "o que acontece?"

Depois de entregar, olhando para dentro do sistema pelo que ele emite: métricas, logs e traces.

PARTE 1

Testes unitários

Rápidos, isolados e baratos: a base de tudo

CONCEITO

Por que testar

- Teste automático é um **programa que confere o seu programa**: roda em segundos, quantas vezes quiser.
- Dá **coragem para mudar**: se algo quebrar, o teste avisa antes do usuário.
- Serve de **documentação viva**: o nome do teste diz a regra ("R3: o 16º dia é recusado").
- Pega bugs que o olho não pega.



Um bug que existiu de verdade

Ao escrever os testes do `TarefaService`, o teste **alternarConclui** falhou: concluir ou reabrir uma tarefa **nunca publicava** o evento no RabbitMQ.

O código compilava e a API respondia 200. Só o teste notou.

Teste não prova que não há bugs. Prova que os bugs que você pensou em procurar não estão lá.

CONCEITO

A pirâmide de testes



mais rápido, mais barato, mais estável

- **Unitário** aponta **onde** quebrou (uma classe, um método).
- **E2E** garante que as peças funcionam **juntas**.
- Um E2E que falha diz que algo quebrou, mas não onde: por isso são poucos.

PRÁTICA

Anatomia de um teste: preparar, agir, conferir

mapper/PrevisaoMapperTest.java

```java
class PrevisaoMapperTest {

    private final PrevisaoMapper mapper = new PrevisaoMapper();

    @Test
    @DisplayName("código desconhecido não quebra")
    void codigoDesconhecido() {
        // preparar: uma resposta com o código 42
        var resposta = resposta(42, 10);

        // agir: chamar o que está sendo testado
        PrevisaoDTO previsao = mapper.toPrevisao(resposta, 1);

        // conferir: o resultado esperado
        assertThat(previsao.descricao())
                .isEqualTo("Código do tempo 42");
        assertThat(previsao.icone()).isEqualTo("?");
    }
}
```

- **Preparar** (arrange): montar os objetos e os dados.
- **Agir** (act): chamar **uma** coisa.
- **Conferir** (assert): comparar com o esperado, com `assertThat` (AssertJ).
- `@DisplayName` descreve a **regra**, não o método.
- Sem Spring, sem banco, sem rede: só `new`.

PRÁTICA

Mock: o dublê que isola a classe



sem internet, sem esperar, sem depender do Open-Meteo

service/PrevisaoServiceTest.java

```
client = mock(OpenMeteoClient.class);                    // o dublê
service = new PrevisaoService(client, new PrevisaoMapper(), metricas);

// quando pedirem a previsão de hoje, o dublê responde isto:
when(client.buscarPrevisao(LAT, LON, hoje, hoje)).thenReturn(respostaPara(hoje));

service.buscar(LAT, LON, hoje);                  // 1ª vez: vai ao client
PrevisaoDTO segunda = service.buscar(LAT, LON, hoje);    // 2ª vez: cache

assertThat(segunda.origem()).isEqualTo("cache");
verify(client, times(1)).buscarPrevisao(LAT, LON, hoje, hoje);    // só UMA chamada
```

REFERÊNCIA

O que mockar e o que não mockar

Dependência	Teste unitário	Teste E2E	Por quê
OpenMeteoClient	Mock	Mock	Serviço de fora: lento, instável e a resposta muda
Repository (banco)	Mock	Real (container)	No unitário só interessa a regra; no E2E, o SQL e as migrations
RabbitTemplate	Mock	Real (container)	Provar que a mensagem passa pelo broker de verdade
PrevisaoMapper	Real	Real	Classe pura e rápida: fingir seria perder o teste

Regra: mocke o que **não é seu** ou é **lento e instável**. O que é seu e barato, use de verdade.

NO PROJETO

Os 31 testes unitários do projeto

Teste	Qtd.	Técnica que mostra
PrevisaoMapperTest	9	Teste puro, @ParameterizedTest para a tabela de códigos, dado de fora (null)
PrevisaoServiceTest	9	Mockito: regras R1 a R3, cache com verify(times(1)), falha que não vai ao cache
TarefaServiceTest	8	Repositórios e publisher mockados: cada ação publica o evento certo; erro não publica
TarefaEventoPublisherTest	2	Exchange e routing key; broker fora do ar não derruba o cadastro
OpenMeteoClientTest	3	MockRestServiceServer: confere a URL e traduz 500 e JSON quebrado em 502

Roda tudo em **menos de 1 segundo**: dá para rodar a cada alteração.

PRÁTICA

Testando o cliente HTTP sem internet

client/OpenMeteoClientTest.java

```
RestClient.Builder builder = RestClient.builder();
servidor = MockRestServiceServer.bindTo(builder).build();
client = new OpenMeteoClient(builder, URL, metricas);

@Test
void erroDoServidor() {
    // o "Open-Meteo" responde 500
    servidor.expect(requestTo(startsWith(URL)))
            .andRespond(withServerError());

    assertThatThrownBy(() -> client.buscarAgora(-28.6, -49.3))
        .isExactlyInstanceOf(PrevisaoIndisponivelException.class)
        .hasMessageContaining("500");
}
```

- O `MockRestServiceServer` **faz o papel do Open-Meteo** dentro da JVM.
- Confere o que o client **enviou**: caminho, `latitude`, `start_date`.
- Decide o que ele **responde**: sucesso, 500, JSON quebrado.
- Assim testamos o **502** sem derrubar nada.

BOAS PRÁTICAS

Teste o caminho feliz e também o erro



Caminho feliz

A previsão de amanhã volta traduzida, com descrição e ícone.



Limites

O 15º dia depois de hoje é aceito; o 16º é recusado (R3).



Dado ruim de fora

Código do tempo desconhecido, chance de chuva null, resposta sem o dia.



Falha do outro lado

Open-Meteo em 500 vira 502; demora demais vira 504; o erro não vai para o cache.

Um teste por regra, com o **nome da regra** no `@DisplayName`: o relatório vira a especificação.

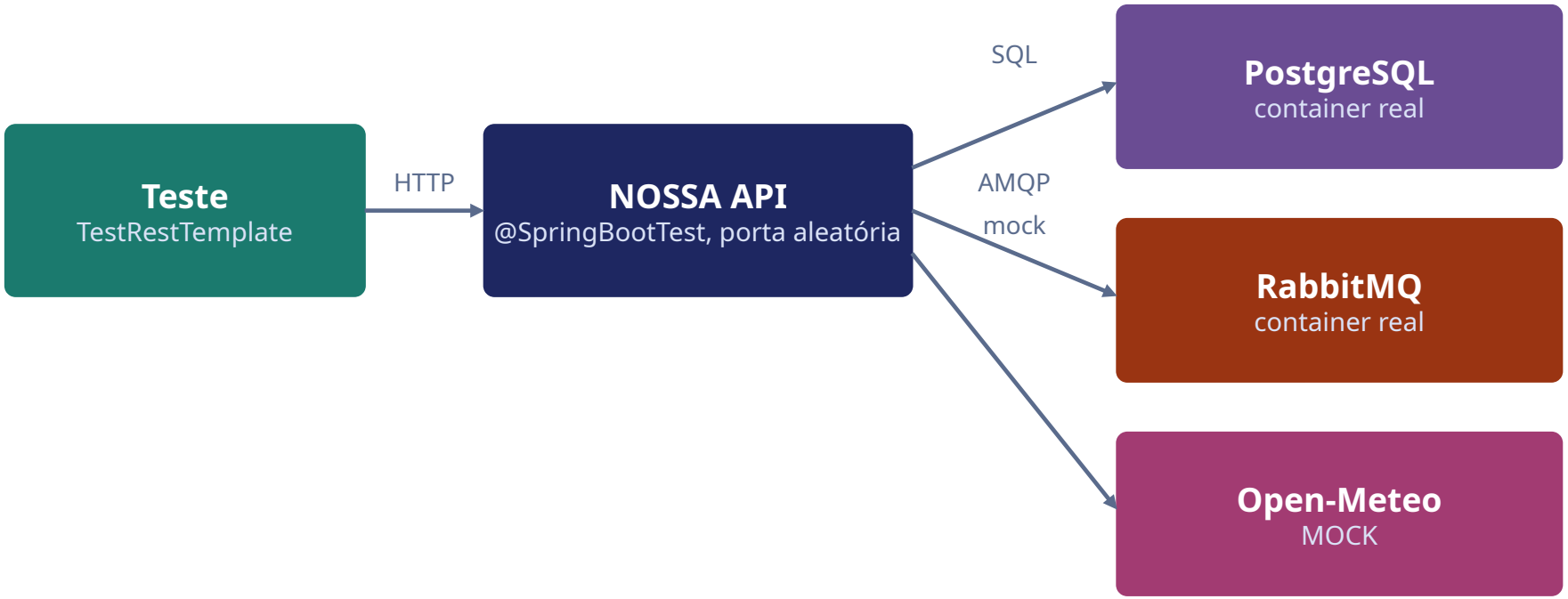
PARTE 2

Testes E2E (ponta a ponta)

A aplicação inteira, com banco e broker de verdade, e a tela no navegador

CONCEITO

O que é um teste E2E



- Passa por **controller, service, JPA, migrations do Flyway e mensageria**.
- É o mais próximo da produção: sem H2 e sem broker falso.
- Só o **Open-Meteo** é simulado (está na internet, fora do nosso controle).

O E2E garante que as peças **funcionam juntas**. Precisa do **Docker** rodando.

PRÁTICA

Testcontainers: infraestrutura de verdade no teste

e2e/AbstractE2ETest.java

```java
@SpringBootTest(webEnvironment = RANDOM_PORT)
@AutoConfigureObservability
abstract class AbstractE2ETest {

    @ServiceConnection
    static final PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @ServiceConnection
    static final RabbitMQContainer rabbit =
        new RabbitMQContainer("rabbitmq:3.13-management-alpine");

    static { postgres.start(); rabbit.start(); }

    // o Open-Meteo é de fora: troca por um mock
    @MockitoBean protected OpenMeteoClient openMeteoClient;

    @Autowired protected TestRestTemplate http;
}
```

- O container sobe **antes** dos testes e some depois.
- `@ServiceConnection` aponta `spring.datasource.*` e `spring.rabbitmq.*` para ele.
- Cada execução parte de um **banco limpo**: sem "funciona na minha máquina".
- `RANDOM_PORT`: a API sobe numa porta livre.

PRÁTICA

O que é assíncrono: espere, não durma

e2e/TarefaApiE2ETest.java

```java
@Test
void eventoPassaPeloRabbit() {
    double antes = metricas
        .counter("tarefas.eventos.consumidos", "tipo", "criada")
        .count();

    criar("E2E: evento no RabbitMQ");   // POST /tarefas de verdade

    // o listener roda em outra thread: espera até 10 s
    await().atMost(Duration.ofSeconds(10)).untilAsserted(() ->
        assertThat(metricas
            .counter("tarefas.eventos.consumidos", "tipo", "criada")
            .count()).isGreaterThan(antes));
}
```

`Thread.sleep(2000)`
Lento quando dá certo e frágil quando a máquina está ocupada.

`await().untilAsserted(...)`
Para assim que a condição vale, e falha com prazo se não valer.

O teste confere o **efeito completo**: HTTP → banco → exchange → fila → listener → métrica.

LIÇÕES

Três armadilhas que encontramos ao escrever estes testes

Containers mortos

Com `@Container` static, o container morre ao fim de **cada classe**, mas o Spring **reaproveita o contexto** nas próximas. Resultado: timeout em tudo.

Solução: container singleton, iniciado uma vez.

Prometheus 404

Em `@SpringBootTest`, a exportação de métricas vem **desligada**: `/actuator/prometheus` dá 404.

Solução: `@AutoConfigureObservability`.

Bug real

Os testes do `TarefaService` mostraram que **concluir ou reabrir** uma tarefa não publicava o evento.

Solução: corrigir o código, e o teste fica de guarda.

NO PROJETO

Rodando os testes

31 **unitários**
sem infraestrutura

14 **E2E**
PostgreSQL e RabbitMQ reais

45 **no total**
0 falhas

terminal

```
$ docker compose --profile testes run --rm testes

[INFO] Tests run: 9, Failures: 0 -- PrevisaoMapperTest
[INFO] Tests run: 9, Failures: 0 -- PrevisaoServiceTest
[INFO] Tests run: 5, Failures: 0 -- TarefaApiE2ETest
[INFO] Tests run: 45, Failures: 0, Errors: 0
```

- Sem script: o Maven roda **dentro de um container**.
- Não precisa subir nada antes: os E2E criam os containers.
- Falhou? O relatório está em `target/surefire-reports`.

PRÁTICA

Rodando sem script: o Maven dentro de um container

Para	Comando
Todos os testes (31 + 14)	docker compose --profile testes run --rm testes
Só os unitários (31)	docker compose --profile testes run --rm testes mvn -B -ntp test "-Dtest=!*E2ETest"
Subir a API	docker compose --profile app up -d api
Ver o log da API	docker compose logs -f api
Parar tudo	docker compose --profile app --profile testes down

- O serviço `testes` monta o projeto e o **socket do Docker**: o Testcontainers cria o PostgreSQL e o RabbitMQ dos testes E2E.
- As dependências ficam num volume (`maven_repo`): só a primeira execução demora.
- Também dá para rodar pela IDE: botão direito em `src/test/java` → Run All Tests.

PRÁTICA

Testes de tela: o Playwright abre o navegador

- O **Playwright** controla um navegador de verdade (Chromium): digita, clica e confere o que aparece.
- Testa a **tela** (`index.html`); o Testcontainers testa a **API**. É o topo da pirâmide.
- **Espera sozinho**: cada ação aguarda o elemento ficar pronto, sem `sleep`.
- A previsão do tempo é **simulada** (`page.route`): sem internet e sem Open-Meteo.
- A localização é **fixa** (Criciúma/SC, no código): a página não pede nada ao navegador.
- 6 testes: abrir, cadastrar, validar, previsão, concluir e excluir.

playwright/tests/cadastro.spec.ts

```
test('cadastra uma tarefa', async ({ page }) => {

  await page.locator('#input-titulo')
            .fill('Estudar Playwright');
  await page.locator('#input-responsavel').fill('Ana');
  await page.locator('#input-prazo').fill('2026-11-04');
  await page.locator('#btn-add').click();

  const item = page.locator('#lista-abertas li',
      { hasText: 'Estudar Playwright' });

  await expect(item).toBeVisible();
  await expect(item).toContainText('Ana');
});
```

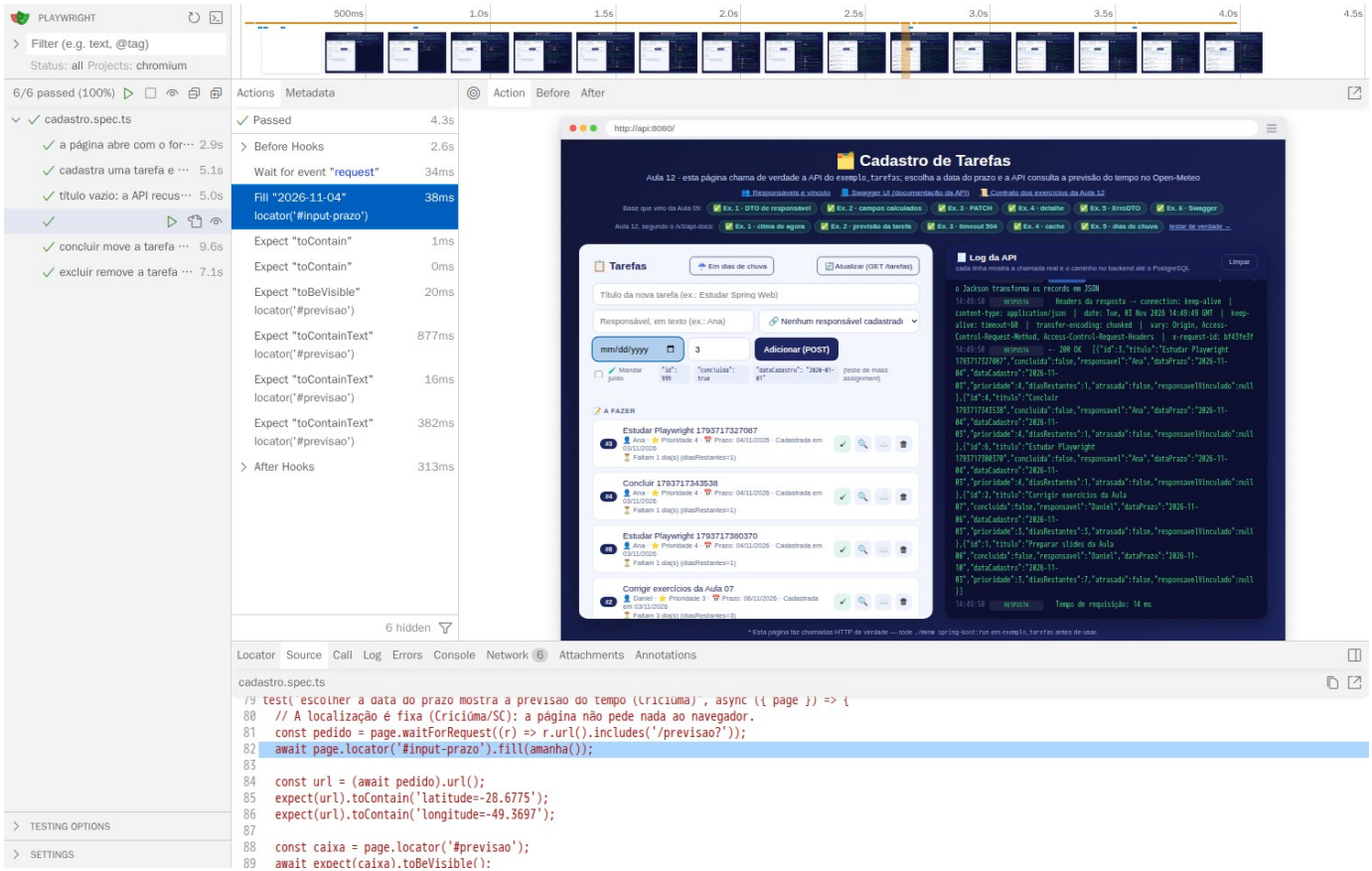
PRÁTICA

Mostrando no navegador: o modo UI do Playwright

terminal

```
docker compose --profile ui up playwright
# depois, abra no navegador:
# http://localhost:8081
```

- ▶️ roda todos os testes. Clique num passo (Fill, Click, Expect) e veja a **tela naquele instante**.
- A linha do tempo mostra cada momento; embaixo, o código, a rede e o console.
- Salvou o teste? O modo watch roda de novo sozinho.



Captura real: 6/6 testes passando e o passo da data do prazo selecionado.

PRÁTICA

Ver os testes rodando sozinhos: 4 formas



Modo UI

docker compose --profile ui up playwright

Abra **localhost:8081** e clique em ▶️. A tela da aplicação muda passo a passo.



Sem tela

docker compose --profile ui run --rm playwright-run

Roda os 6 testes em segundo plano e mostra só o resultado no terminal.



Testes Java

docker compose --profile testes run --rm testes

Os 45 testes não têm tela: acompanhe pelos logs no terminal.



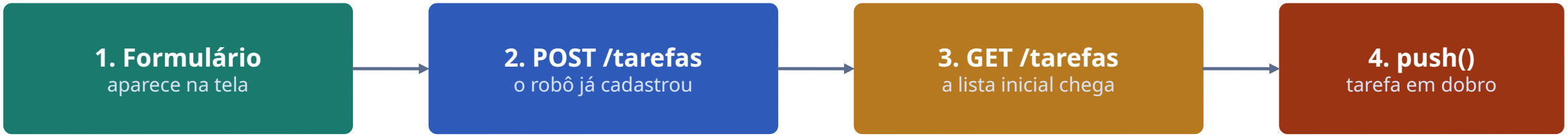
Grafana

Abra **localhost:3000** durante os testes.

Os gráficos de requisições e eventos do RabbitMQ sobem enquanto eles executam.

No modo UI, passe o mouse na linha do tempo para ver a tela antes e depois de cada ação; as abas Network e Console mostram as chamadas.

O robô achou um bug que ninguém tinha visto



- O formulário aparece **antes** de a lista inicial terminar de carregar.
- Se a tarefa é cadastrada nesse intervalo, a lista que chega **já a contém**, e o `tarefas.push(criada)` a insere **de novo**: a mesma `#id` aparece duas vezes.
- O Playwright acusou na hora (strict mode violation: 2 elementos). Uma pessoa dificilmente veria.
- No teste: `waitForResponse` espera a lista antes de agir. Na página, a correção seria não inserir se o `id` já estiver em `tarefas`.

PARTE 3

Observabilidade

Descobrir o que o sistema está fazendo, olhando o que ele emite

CONCEITO

O que é observabilidade: três sinais



Métricas

Números ao longo do tempo.

Quanto? Com que frequência? Está piorando?

Barato e agregado: ótimo para gráficos e alertas.



Logs

Eventos com texto e contexto.

O que aconteceu naquela requisição?

Detalhado: ótimo para investigar um caso.



Traces

O caminho de uma requisição entre serviços.

Por onde passou e onde demorou?

Próximo passo do projeto (OpenTelemetry).

Hoje: **métricas** (Prometheus + Grafana) e **logs** com `requisicaoId`. Traces ficam como evolução.

CONCEITO

A arquitetura da nossa stack



- A API **expõe** as métricas em texto: o Actuator com o Micrometer faz isso.
- O Prometheus **coleta** (scrape) e guarda a história: o que era agora vira tendência.
- O Grafana **desenha**: o dashboard já vem pronto, provisionado por arquivo.
- Tudo no `docker-compose.yml`, junto com o PostgreSQL e o RabbitMQ.

PRÁTICA

Actuator: a aplicação mostra como está

pom.xml

```
<dependency>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

application.properties

```
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always
management.metrics.tags.application=exemplo-tarefas
spring.application.name=exemplo-tarefas
```

GET /actuator/health

```
{
  "status": "UP",
  "components": {
    "db":       { "status": "UP",
                  "details": { "database": "PostgreSQL" } },
    "rabbit":   { "status": "UP",
                  "details": { "version": "3.13" } },
    "diskSpace": { "status": "UP" }
  }
}

// Um status só (UP/DOWN) que resume
// banco + broker + disco: é o que o
// orquestrador consulta para saber se
// a instância está saudável.
```

PRÁTICA

Micrometer: as métricas do nosso negócio

messaging/TarefaEventoPublisher.java

```
try {
    rabbitTemplate.convertAndSend(EXCHANGE, "tarefa." + tipo, evento);
    metricas.counter("tarefas.eventos.publicados",
            "tipo", tipo, "resultado", "ok").increment();
} catch (AmqpException e) {
    metricas.counter("tarefas.eventos.publicados",
            "tipo", tipo, "resultado", "falha").increment();
}

// client/OpenMeteoClient.java: quanto demora cada chamada
Timer.Sample amostra = Timer.start(metricas);
try {
    return chamada.get();
} finally {
    amostra.stop(metricas.timer("openmeteo.chamadas",
            "resultado", resultado));
}
```

- `MeterRegistry` é injetado como qualquer bean.
- **Etiquetas** (`tipo`, `resultado`) permitem separar sucesso de falha **na mesma métrica**.
- Nome com ponto vira `tarefas_eventos_publicados_total` no Prometheus.
- `tarefas.operacoes{operacao}` conta cada operação de tarefa que deu certo (criar, concluir, remover...).
- Cuidado com etiqueta de alta cardinalidade (id de tarefa, e-mail): cada valor vira uma série nova.

REFERÊNCIA

Os tipos de métrica

Tipo	O que é	Exemplo no projeto	Pergunta que responde
Counter	Só cresce (reinicia no restart)	tarefas.eventos.publicados	Quantas vezes aconteceu? Qual a taxa por segundo?
Gauge	Um valor que sobe e desce	hikaricp.connections.active	Quantos há agora?
Timer	Contagem + duração	openmeteo.chamadas	Quantas chamadas e quanto demoram?
Histogram	Distribuição por faixas (buckets)	http.server.requests	Qual o p95 e o p99 da latência?

Média mente. Se 99 requisições levam 10 ms e uma leva 10 s, a média parece boa. Por isso usamos **percentis** (p95, p99).

PRÁTICA

Prometheus: coletar e consultar

observabilidade/prometheus/prometheus.yml

```yaml
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: exemplo-tarefas
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: ["host.docker.internal:8080"]
```

- A API roda no host; o Prometheus, no container: `host.docker.internal`.
- `rate()` transforma um contador em **taxa por segundo**.

PromQL (aba Graph do Prometheus)

```
# requisições por segundo, por rota
sum by (uri) (
  rate(http_server_requests_seconds_count[1m]))

# latência p95
histogram_quantile(0.95,
  sum by (le, uri) (
    rate(http_server_requests_seconds_bucket[1m])))

# eventos publicados x consumidos
rate(tarefas_eventos_publicados_total[1m])
rate(tarefas_eventos_consumidos_total[1m])

# taxa de erro do Open-Meteo
rate(openmeteo_chamadas_seconds_count
       {resultado="erro"}[1m])
```

PRÁTICA

O dashboard: os quatro sinais de ouro

Tráfego

Quantas requisições por segundo, por rota.
`http_server_requests_seconds_count`

Latência

Quanto demora: o p95 por rota.
`http_server_requests_seconds_bucket`

Erros

4xx, 5xx, 502 e 504 por segundo, mais as falhas do Open-Meteo.

Saturação

O quanto dos recursos está em uso: memória heap e conexões do pool do banco.

Mais painéis: todas as rotas (tabela), erros por rota, operações de tarefa, Open-Meteo, cache e RabbitMQ.

PRÁTICA

Toda rota da API é observada

Métricas HTTP

Toda requisição, com a rota como etiqueta: `/tarefas/{id}`, e não `/tarefas/7`. Rota nova aparece sozinha.

Métrica de negócio

`tarefas_operacoes_total{operacao}`: criar, atualizar, concluir, reabrir, remover, vincular...

Log por requisição

Uma linha em toda rota: método, rota, status e duração, com o `requisicaoId`.

console

```
INFO [313cd498] POST /tarefas -> 201 (34 ms)
INFO [0c38c526] GET /tarefas -> 200 (4 ms)
INFO [c7bce90f] GET /responsaveis/{id} -> 200 (24 ms)
INFO [a91f03d2] PUT /tarefas/{id}/concluir -> 200 (12 ms)
INFO [77be10c4] GET /tarefas/999999 -> 404 (3 ms)
```

19 rotas em 3 controllers (tarefas, responsáveis e previsão)

O `CoberturaObservabilidadeE2ETest` chama as 19 e falha se alguma não aparecer nas métricas.

PRÁTICA

Logs: seguir uma requisição do começo ao fim

observability/RequisicaoIdFilter.java

```java
@Component
public class RequisicaoIdFilter extends OncePerRequestFilter {

    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain) ... {
        String id = request.getHeader("X-Request-Id");
        if (id == null || id.isBlank()) {
            id = UUID.randomUUID().toString().substring(0, 8);
        }
        MDC.put("requisicaoId", id);
        response.setHeader("X-Request-Id", id);
        long inicio = System.nanoTime();
        try {
            chain.doFilter(request, response);
        } finally {
            registrarAcesso(request, response, inicio);
            MDC.remove("requisicaoId");   // a thread volta ao pool
        }
    }
    // uma linha por requisição, em TODA rota (menos /actuator)
    log.info("{} {} -> {} ({} ms)", metodo, rota, status, ms);
}
```

console

```
INFO [cd6976cb] Evento publicado:
     tarefa.criada (tarefa 7)
INFO [cd6976cb] POST /tarefas -> 201 (34 ms)
INFO [c7bce90f] GET /tarefas/{id} -> 200 (4 ms)
INFO [a91f03d2] GET /tarefas/999 -> 404 (3 ms)
```

- `logging.pattern.level=%5p [%X{requisicaoId:-}]` põe o id em **toda** linha.
- Uma linha por requisição, em **toda rota**: método, **rota** (`/tarefas/{id}`), status e duração.
- O cliente pode mandar o próprio `X-Request-Id`.

PRÁTICA

A stack inteira no docker-compose

docker-compose.yml

```yaml
services:
  postgres:    # :5441   o banco
  rabbitmq:    # :5673   broker (painel :15673)
  prometheus:  # :9090   coleta as métricas
  grafana:     # :3000   dashboards

  # sem script: só sobem com --profile
  testes:      # profile "testes"
    image: maven:3.9-eclipse-temurin-25
    volumes:
      - .:/app
      - /var/run/docker.sock:/var/run/docker.sock
    command: mvn -B -ntp test

  api:         # profile "app"
    image: maven:3.9-eclipse-temurin-25
    ports: ["8080:8080"]
    command: mvn -B -ntp spring-boot:run
```

- **Provisionamento**: datasource e dashboard vêm de arquivos versionados, sem clicar em nada.
- O dashboard é um **JSON no Git**: revisável e reproduzível.
- `profiles` (`testes`, `app`, `carga`): serviços que só sobem quando pedidos.

⚠️ **Só para aula:** o Grafana está com acesso anônimo de admin e o `/actuator` está aberto. Em produção, não.

PARTE 4

Na prática

Subir tudo, gerar carga e ler os gráficos

PRÁTICA

Teste de carga na fila do RabbitMQ

terminal

```
# PowerShell: $env:CARGA_TAXA=300; $env:CARGA_PRODUTORES=2
$ CARGA_TAXA=300 CARGA_PRODUTORES=2 \
    docker compose --profile carga run --rm perftest

sending rate avg: 641 msg/s
receiving rate avg: 0 msg/s    # quem consome é a API
tarefas.notificacoes  0 mensagens  1 consumidor
```

~1.050 — **msg/s publicadas**
2 produtores × 500 msg/s

~4.800 — **mensagens**
consumidas pela API

0 — **backlog no fim**
o listener acompanhou

- Com a API **desligada**, as mensagens se **acumulam** (backlog). Ao ligar, a fila esvazia.
- No Grafana: o painel **publicado x consumido** mostra se o consumo acompanha.
- Perguntas de um teste de carga: até que taxa o consumidor aguenta? Onde está o gargalo?

PRÁTICA

Teste de carga sem script: só o docker compose

Variável	Padrão	O que é
CARGA_TAXA	200	msg/s por produtor
CARGA_SEGUNDOS	60	Duração do teste
CARGA_PRODUTORES	1	Quantos produtores
CARGA_CONSUMIDORES	0	0 = a API consome

PowerShell (Windows)

```
$env:CARGA_TAXA=500
$env:CARGA_SEGUNDOS=60
$env:CARGA_PRODUTORES=2
docker compose --profile carga run --rm perftest
```

bash (Linux / Mac)

```
CARGA_TAXA=500 CARGA_PRODUTORES=2 \
  docker compose --profile carga run --rm perftest
```

- O RabbitMQ cria **exchange, fila, binding e usuário** ao subir (`carga/definitions.json`): roda até com a API desligada.
- Para ver o que sobrou na fila: painel do RabbitMQ (aba Queues).

Ao carregar um arquivo de definições, o RabbitMQ **não cria mais o usuário padrão**: o usuário `tarefas` precisa estar no próprio arquivo.

PRÁTICA

Teste de carga completo: alimenta todos os gráficos

terminal

```
docker compose --profile carga up \
  --attach carga-http --attach perftest \
  carga-http perftest
```

- **k6** (`carga/http.js`): cada iteração cria uma tarefa e percorre as **19 rotas**, com **400 e 404** de propósito.
- **PerfTest**: enche a fila do RabbitMQ.
- Só HTTP: `run --rm carga-http`. Variáveis: `CARGA_VUS`, `CARGA_SEGUNDOS`, `CARGA_TAXA`.

Painel do Grafana	Quem alimenta
Requisições, latência, rotas	as 19 rotas (k6)
Erros 4xx e 5xx	400 e 404 provocados
Open-Meteo e cache	rotas de previsão
RabbitMQ publicados x consumidos	HTTP + PerfTest
Operações de tarefa	criar, concluir, reabrir...
Memória e pool do banco	efeito da carga

A previsão usa o Open-Meteo de verdade (precisa de internet). Sem internet, vira erro 5xx, que também aparece nos painéis.

PRÁTICA

Roteiro da demonstração

1 Rodar os testes

`docker compose --profile testes run --rm testes`: 45 verdes

2 Subir a stack

```
docker compose up -d
```

3 Subir a API

```
docker compose --profile app up -d api
```

4 Conferir o scrape

Prometheus → Status → Targets: UP

5 Gerar movimento

```
docker compose --profile carga up --attach carga-http --attach perftest carga-http perftest
```

6 Ler o dashboard

Grafana → Aula 13 → Exemplo Tarefas

REFERÊNCIA

Onde olhar

Endereço	O que é
`localhost:8080/actuator/health`	A aplicação, o banco e o RabbitMQ estão de pé?
`localhost:8080/actuator/prometheus`	Todas as métricas, em texto, como o Prometheus as lê
localhost:9090	Prometheus: Status → Targets (UP) e a aba Graph para PromQL
localhost:3000	Grafana (sem login): pasta Aula 13 → Exemplo Tarefas: visão geral
localhost:15673	Painel do RabbitMQ (usuário e senha: tarefas): filas e taxas

As portas são diferentes das da Aula 12 (banco `5441`, RabbitMQ `5673`) para rodar as duas lado a lado.

FECHAMENTO

O que levamos desta aula

Muitos unitários, poucos E2E

A pirâmide: base rápida e barata, topo lento e valioso.

Mocke o que não é seu

O Open-Meteo vira dublê; banco e broker ficam reais no E2E.

Métricas contam a história

Tráfego, latência, erros e saturação, mais as do negócio.

Log com correlação

Um `requisicaoId` em cada linha: dá para seguir uma requisição.

"Funciona?" → teste automático. **"O que está acontecendo?"** → observabilidade.

PRÓXIMOS PASSOS

Para ir além



Traces

OpenTelemetry + Tempo ou Jaeger: ver por onde a requisição passou e onde demorou.



Alertas

Alertmanager: avisar quando o erro ou a latência passarem de um limite.



Testes no CI

Rodar `mvn test` a cada push (GitHub Actions) e bloquear o merge se falhar.



Cobertura

JaCoCo: ver quais linhas os testes não exercitam, sem tratar a % como meta.

ENCERRAMENTO

Obrigado!

Material: Aula 13/exemplo_tarefas_resolvido (README, src/test e observabilidade/). Contato: daniel.placido@satc.edu.br